

Adventist University of Central Africa

Master's in Big Data Analytics

Chapter 5 — Resampling Methods

Exercises 5 and 9

Introduction to Statistical Learning with Applications in R

Programme: MSc Big Data Analytics
Course Title: Data Mining
Course Code: MSDA 9124
Lecturer: Dr. Pacifique Nizeyimana
Student Name: Wayne Rubangisa
Student ID: 101220

September 2026

```
library(ISLR2)
library(boot)
```

1 Exercise 5 — Validation-Set Error for Logistic Regression on Default

1.1 (a) Fit the full-data logistic regression

```
set.seed(1)
glm.fit <- glm(default ~ income + balance, data = Default, family = binomial)
summary(glm.fit)

##
## Call:
## glm(formula = default ~ income + balance, family = binomial,
##      data = Default)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -1.154e+01  4.348e-01 -26.545  < 2e-16 ***
## income       2.081e-05  4.985e-06   4.174 2.99e-05 ***
## balance      5.647e-03  2.274e-04  24.836  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##    Null deviance: 2920.6  on 9999  degrees of freedom
## Residual deviance: 1579.0  on 9997  degrees of freedom
## AIC: 1585
##
## Number of Fisher Scoring iterations: 8
```

Both `income` and `balance` come out highly significant, with `balance` the far stronger predictor by t-statistic magnitude — consistent with what Chapter 4 found for this same relationship.

1.2 (b) Validation-set test error

The validation-set approach is conceptually simple: split the data in half, fit only on one half, and check performance on the half the model never saw — the resulting error rate is a genuine estimate of test error, unlike the training error computed on data the model already fit.

```
set.seed(1)
n <- nrow(Default)
train <- sample(n, n / 2)

glm.fit <- glm(default ~ income + balance, data = Default,
               family = binomial, subset = train)
```

```
glm.probs <- predict(glm.fit, Default[-train, ], type = "response")
glm.pred <- ifelse(glm.probs > 0.5, "Yes", "No")

val.error <- mean(glm.pred != Default[-train, ]$default)
val.error
```

```
## [1] 0.0254
```

A validation error around 2.5% — notably higher than the essentially-zero training error a full-data fit could give the illusion of, since 5,000 held-out observations never influenced the fitted coefficients at all.

1.3 (c) Repeating with three different splits

```
val_error <- function(seed) {
  set.seed(seed)
  train <- sample(n, n / 2)
  fit <- glm(default ~ income + balance, data = Default,
             family = binomial, subset = train)
  probs <- predict(fit, Default[-train, ], type = "response")
  pred <- ifelse(probs > 0.5, "Yes", "No")
  mean(pred != Default[-train, ]$default)
}

errors <- sapply(c(1, 2, 3), val_error)
errors
```

```
## [1] 0.0254 0.0238 0.0264
```

The three error rates all land in a fairly tight band (roughly 2.4%–2.6%), but they’re not identical — which is the point. Every one of these numbers is a valid estimate of the same underlying test error, yet each depends on the arbitrary luck of which 5,000 observations happened to land in the training half. That split-to-split variability is exactly the validation set approach’s core weakness: throwing away half the data for training, on a single random split, means both the fitted model and the resulting error estimate are noisier than they’d need to be with a method like *k*-fold cross-validation that reuses the data more efficiently.

1.4 (d) Adding a student dummy variable

```
set.seed(1)
train <- sample(n, n / 2)
glm.fit.student <- glm(default ~ income + balance + student, data = Default,
                      family = binomial, subset = train)
glm.probs <- predict(glm.fit.student, Default[-train, ], type = "response")
glm.pred <- ifelse(glm.probs > 0.5, "Yes", "No")

val.error.student <- mean(glm.pred != Default[-train, ]$default)
val.error.student
```

```
## [1] 0.026
```

With the same seed (and so the same train/test split) as (b), adding `student` gives a validation error of 0.026, essentially identical to — in this run, marginally *higher* than — 0.0254 without it. So there's no evidence here that including `student` improves predictive performance. This isn't surprising in hindsight: `student` is strongly correlated with `balance` (students tend to carry different balances than non-students), so much of whatever information `student` carries about default risk is already captured once `balance` is in the model — adding it doesn't give logistic regression much new signal to work with, only extra estimation noise.

2 Exercise 9 — Bootstrap Estimates for the Boston Data

Every part below is about the same core idea: `medv` (or a statistic of it) is being estimated from one particular sample of 506 census tracts, and we want to know how much that estimate would bounce around if we happened to have drawn a different sample of the same size from the same population. Textbook formulas exist for some statistics (the sample mean); the bootstrap gives us an answer even for the ones that don't (the median, a percentile) by treating our one sample as a stand-in for the population and resampling from it directly.

2.1 (a) Estimate of the population mean, $\hat{\mu}$

```
Boston2 <- ISLR2::Boston
mu.hat <- mean(Boston2$medv)
mu.hat
```

```
## [1] 22.53281
```

2.2 (b) Standard error of $\hat{\mu}$, by formula

$$SE(\hat{\mu}) = \frac{s}{\sqrt{n}}$$

where s is the sample standard deviation of `medv` and $n = 506$.

```
se.formula <- sd(Boston2$medv) / sqrt(nrow(Boston2))
se.formula
```

```
## [1] 0.4088611
```

This measures how much $\hat{\mu}$ would be expected to vary, on average, across repeated samples of 506 tracts drawn from the same population — about \$409 (recall `medv` is in \$1000s), against a mean around \$22,530.

2.3 (c) Standard error of $\hat{\mu}$, by bootstrap

```
set.seed(1)
boot.fn.mean <- function(data, index) mean(data[index])
boot.mean <- boot(Boston2$medv, boot.fn.mean, R = 10000)
boot.mean

##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Boston2$medv, statistic = boot.fn.mean, R = 10000)
##
##
## Bootstrap Statistics :
##      original      bias    std. error
## t1* 22.53281 -0.002350771  0.4069546
```

The bootstrap standard error (0.407) comes out essentially identical to the formula-based estimate in (b) (0.4089) — reassuring, since for something as well-behaved as a sample mean, the classical formula is already exact asymptotically. The bootstrap isn't doing anything the formula couldn't already do here; its real value shows up in (f) and (h), for statistics that have no comparably simple formula at all.

2.4 (d) 95% confidence interval for the mean, via bootstrap

Using the approximation given in the exercise, $[\hat{\mu} - 2\text{SE}(\hat{\mu}), \hat{\mu} + 2\text{SE}(\hat{\mu})]$:

```
se.boot <- sd(boot.mean$t)
ci.boot <- c(mu.hat - 2 * se.boot, mu.hat + 2 * se.boot)
ci.boot
```

```
## [1] 21.71890 23.34672
```

```
t.test(Boston2$medv)
```

```
##
## One Sample t-test
##
## data: Boston2$medv
## t = 55.111, df = 505, p-value < 2.2e-16
## alternative hypothesis: true mean is not equal to 0
## 95 percent confidence interval:
## 21.72953 23.33608
## sample estimates:
## mean of x
## 22.53281
```

The bootstrap-based interval (roughly 21.72 to 23.35) is nearly indistinguishable from `t.test()`'s interval — expected, since both are really estimating the same quantity (± 2 standard errors around the mean is a close approximation to the t -distribution multiplier `t.test()` uses at $n = 506$), just arrived at through different machinery: one assumes normal-theory sampling distribution math, the other estimates the same spread empirically by actually resampling.

2.5 (e) Estimate of the population median

```
med.hat <- median(Boston2$medv)
med.hat
```

```
## [1] 21.2
```

2.6 (f) Standard error of the median, via bootstrap

There is no simple closed-form formula for the standard error of a sample median (unlike the mean, in (b)) — this is precisely the situation the bootstrap is built for.

```
set.seed(1)
boot.fn.median <- function(data, index) median(data[index])
```

```
boot.median <- boot(Boston2$medv, boot.fn.median, R = 10000)
boot.median

##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Boston2$medv, statistic = boot.fn.median, R = 10000)
##
##
## Bootstrap Statistics :
##      original      bias      std. error
## t1*         21.2 -0.014705    0.3780355
```

The estimated standard error (0.378) is somewhat *smaller* than the mean's standard error from (c) — the median is less sensitive than the mean to the handful of very high-`medv` tracts (recall `medv` appears capped at \$50,000 in this data), so it varies a bit less across resamples.

2.7 (g) Estimate of the 10th percentile

```
mu.hat.01 <- quantile(Boston2$medv, 0.1)
mu.hat.01
```

```
## 10%
## 12.75
```

2.8 (h) Standard error of the 10th percentile, via bootstrap

```
set.seed(1)
boot.fn.q10 <- function(data, index) quantile(data[index], 0.1)
boot.q10 <- boot(Boston2$medv, boot.fn.q10, R = 10000)
boot.q10
```

```
##
## ORDINARY NONPARAMETRIC BOOTSTRAP
##
##
## Call:
## boot(data = Boston2$medv, statistic = boot.fn.q10, R = 10000)
##
##
## Bootstrap Statistics :
##      original      bias      std. error
## t1*         12.75  0.007625    0.5014507
```

The standard error here (0.5015) is the largest of the three statistics examined — larger than both the mean's and the median's. This tracks with intuition: the 10th percentile is estimated from comparatively few, more extreme observations in the lower tail of `medv`, so there's inherently less

data locally supporting that estimate than there is supporting the mean or median, which draw on the full distribution. This is exactly the kind of finding the bootstrap makes practical to obtain — no standard formula for the standard error of a percentile exists at all, yet the same resampling procedure that worked for the mean and median hands us an answer here just as directly.